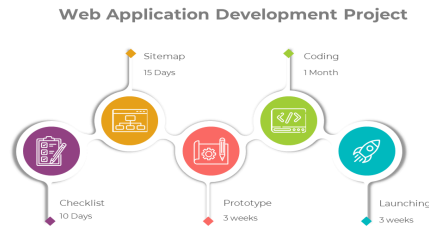


App Dev 2 Project Report



Student Profile

Name: RITIK SONI

Roll Number: BS23F3003280

Email: 23f3003280@ds.study.iitm.ac.in

About Me: I am currently pursuing the BS Degree Program at the **Indian Institute of Technology Madras**, where I am developing a strong foundation in technology and data-driven problem solving. I have a keen interest in web application development and enjoy creating applications that blend analytics, learning, and seamless user experience. I am driven by curiosity and a desire to build impactful, real-world digital solutions.

Project Details

Project Title: Quantified Self App

Context:

This project was developed as part of a placement drive to demonstrate practical knowledge of functional web application and data visualization.

Problem Statement:

The goal was to design and implement a web-based application that helps users systematically track and analyze their work on Placement Cell. The focus was on encouraging self-improvement through data-driven insights.

Approach

This application was developed using Flask with an industry-oriented, scalable architecture aligned with placement-ready development standards. The system focuses on clean backend design, efficient database integration, and optimized data handling practices. Users can log daily activities, with data processed and visualized through interactive dashboards for trend analysis. Structured coding practice, practical implementation of data visualization.

AI/LLM Declaration

I used the free version of ChatGPT to assist with writing Chart.js, functools (wraps), werkzeug.security(for password) and to get suggestions for making the code more structured. Around 20–25% of the work involved using AI for JavaScript guidance, and improving code

reliability. All main coding, logics, debugging, testing, and full integration of the project were done entirely by me.

Technologies and Frameworks Used

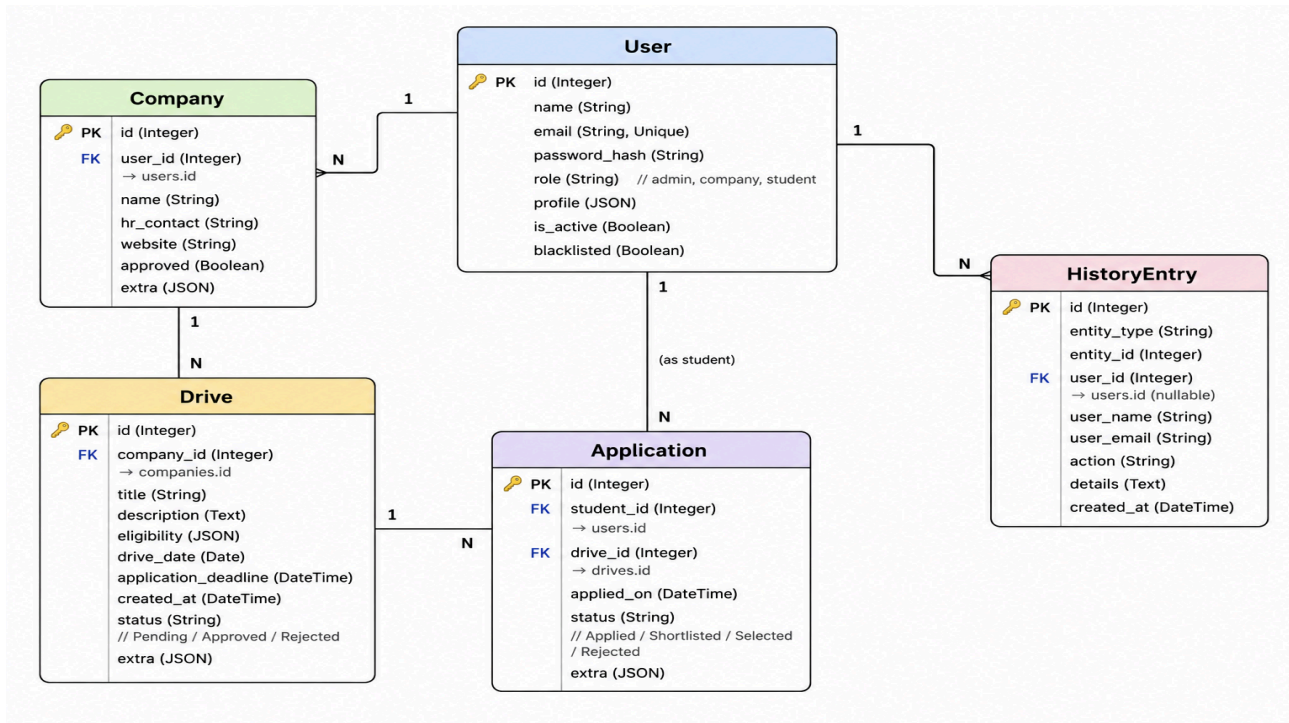
Technology Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Entry template rendering
Bootstrap 5	Frontend styling and responsive design
Chart.js	To visualize trends such as activity vs. time
Flask-Login/ JWT	Authentication and role-based access
Redis	Caching
Celery	Scheduled and asynchronous background jobs
werkzeug.security	These functions are used for securely handling passwords
SQLite	Lightweight local database for storing user data

Database Schema / ER Diagram

Tables:

1. **User** — stores user profile details (id, username, email, password, role [admin,student,company], approved, blocked , status [Pending,rejected,approved] ,created on)
2. **Company** — logs user activities (id, user_id, username, email, company name ,hr name, hr email, industry type, password, approved(by admin only), status, created on)
3. **Student** – Stores student profile details.
4. **Application**—students applied for job (application_id, company_id, student_id, session_id, status, description, application_date)
5. **Notification** – Stores reminders and alerts.
6. **PlacementDrive** – Stores recruitment drive details.

Relationships: Showing the table structure.



API Resource Endpoints

Endpoint	Method	Description
<code>/api/login</code>	POST	Authenticate user and generate session
<code>/api/register/student</code>	POST	Student registration
<code>/api/drives</code>	GET	View approved placement drives
<code>/api/drive/create</code>	POST	Create placement drive
<code>/api/application/apply</code>	POST	Apply for a placement drive
<code>/api/admin/dashboard</code>	GET	Admin dashboard statistics
etc.		

Architecture and Features (optional)

Architecture Overview:

Backend

- **app.py** – main Flask application entry point
- **models.py** – database models using SQLAlchemy

- **celery_worker.py** – Configures and starts the Celery worker for executing background tasks asynchronously.
- **seed.py** – Populates the database with initial data, such as creating the default administrator account, sample users, or demo products.
- **tasks.py** – Contains Celery background tasks such as sending emails, generating reports, and exporting data.

Frontend

- **index.html** – Main Vue.js application page containing the user interface.
- **app.js** – Vue.js application logic, including user authentication, API requests, and UI interaction.
- **style.css** – Stylesheet for the application's user interface.
- **App.vue** – Root Vue component that serves as the main container for the application.
- **components/** – Contains reusable UI components.
- **services/** – Contains modules responsible for communicating with the Flask backend APIs, handling HTTP requests, authentication services, and reusable API functions.

Implemented Features:

- Admin, Company, and Student authentication
- Company approval workflow
- Placement drive management
- Student application management
- Placement history tracking
- Role-based access control
- Redis caching
- Scheduled reminders
- Monthly placement reports
- CSV export using Celery
- E-mail alert

Additional Features:

- Responsive Bootstrap interface
- Trackable History.
- Search and filtering
- Eligibility validation
- Prevention of duplicate applications
- Chart-based placement statistics
- Easy UI

Video Presentation

Drive link: https://drive.google.com/file/d/1rbUu3fL1Xt1um7ZYPbihw-Bt1H3jl3nl/view?usp=drive_link

Thanks For Reading !!